ASSESSMENT BRIEF

Programme	MSc Computing		
Module Title	SOFTWARE DEVELOPMENT 1		
Module Code	CMP-L001-0		
Module Level	7	**Assessment Type(s)**	CW2 (Code)
Word Length / Duration		**% contribution to module mark**	30%
Deadline (date & time) for Submission	19 Dec. 2025 at 6:00 PM	**Format/Location of submission**	Code/ GodeGrade
Assessment Feedback date:	Instance feedback through CodeGrade.		

Learning Outcomes

Through these assignments, you will learn how to:

✓ Design, implement, test, and debug programs using fundamental programming concepts.

✓ Analyse simple programs and explain their behaviour.

✓ Compare and evaluate different programming solutions for a given problem.

✓ Write Python programs that solve real-world problems using structured programming techniques.

Employability and Professional Skills

This assessment has been designed to provide you with an opportunity to demonstrate your achievement of the following skills, which are critical in a professional context and in jobs.

By completing this assessment, you will be able to:

1. Decompose a complex problem into smaller, manageable components by designing and implementing modular, reusable functions.
2. Write robust, professional-grade code that includes comprehensive input validation, error handling, and addresses logical edge cases.
3. Select and utilise appropriate data structures to effectively store, manage, and process collections of data.

	4. Perform essential data handling operations, including reading data from text files (data ingestion), processing it, and writing formatted output to new files (data persistence). 5. Integrate and leverage third-party libraries (e.g., datetime for date manipulation and tabulate for data presentation) to enhance program functionality. 6. Refactor and extend an existing codebase to add significant new features, a common task in professional software development. 7. Develop configurable and generalised solutions that can be adapted to different user requirements (as seen in the setup_module task). 8. Apply systematic debugging and testing to ensure your program is correct, reliable, and meets all specifications. 9. Communicate your solution effectively by writing clear, well-commented code and documentation that adheres to industry standards (PEP-8).

Assessment Requirements

To complete this coursework, you will design the behaviour of complex programs involving the fundamental programming constructs – functions, operations, and file reading/writing. A text file is provided to complete the tasks. *Time in class can be used to troubleshoot and get feedback on your program*.

The university that hired our Software Development company is very pleased with the outcome of Coursework 2 and has requested an extension.

You are asked to add the following functionality:

1. **Functions (8 points):** Split and pack up your Coursework 1 code into functions. The program must contain a **main**() function and at least two other functions called "**calculate_overall_score**()" and "**determine_category**()".

2. **Input expansion (8 points):** Adjust your program to ask users to enter a student ID (2-digit number), name and date-of-birth (D.o.B) by sequence, before asking for the 4 component scores from Coursework 1 (Coursework1, Coursework 2, Coursework 3 and Final Exam). D.o.B should be in ISO format YEAR-MONTH-DAY.

3. **Loop your code (8 points):** Your program should use a loop to ask for the information of three students, or until the input for user ID is "end".

4. Calculate the students' overall scores and store them for later use (2 points).

5. Determine the students' categories and store them for later use (2 points).

6. **Calculate students' ages (8 points):** According to each user's year of birth, your program should calculate the age of the student and store it for later use. Hint: the library datetime can be used here to make your job easier.

7. **Tabulate (8 points):** Display a table that summarises the input data (ascending order with user ID) using the tabulate library. It should look like:

UID	Name	D.o.B	Age	Score	Category
02	Cristiano Ronaldo	1985-02-05	37	73.3333	First
13	Lieke Martens	1992-12-16	29	86.6667	Upper-First
24	Lucy Bronze	1991-10-28	31	50	Third

8. **File writing (8 points):** Save the table into a new local file named "students.txt"

9. **Make sure that you:** Implement robust input validation for a student's ID, D.o.B., and scores. Your program should give:

a. Warning that says "The input you entered was invalid" for any input errors (6 points)

b. Display use of appropriate data structures for storing student information (5 points)

c. Display use of appropriate code structures for implementing the functionality (5 points)

10. Implement a function called `round_to_category(score)` that takes a numerical score and rounds it to the nearest category. This function should (10 points):

a. Accept a float or integer score as input.
b. Determine the two nearest category boundaries.
c. Round the score to the nearest category based on which boundary it's closer to.
d. Return the rounded score and the corresponding category.

For example:

- If the input score is 73.7, it should return (75, "First") because 73.7 is closer to 75 than to 72.
- If the input score is 69.2, it should return (68, "2:1") because 69.2 is closer to 68 than to 72.

Your implementation should handle edge cases, such as scores exactly halfway between two categories (**round up in these cases**), and scores at or beyond the extremes of the scale.

UID	Name	D.o.B	Age	Raw Score	Rounded Score	Category
02	Cristiano Ronaldo	1985-02-05	37	73.3333	75	First
13	Lieke Martens	1992-12-16	29	86.6667	85	Upper-First
24	Lucy Bronze	1991-10-28	31	50	52	Third

11. **Distinction Advanced Function:** Once everything in requirements 1-10 has been completed, for a distinction grade, extend your program with one more function called advanced(). This function should:

a. Be a complimentary main() method that, instead of taking user input, uses the provided file as input.

b. Accepts in its argument (filename) the name of the text file to read and weights as a list of components' weights.

c. Read the student information record from the provided file "StudentData.txt", which contains student IDs, names, and date-of-births. The function should then use this data as the input for the program.

d. All the other functions used and produces a text file output (requirement 8).

12. Advanced Generalization Function (Distinction level): Implement a function called `setup_module()` that allows the user to define the assessment criteria for any module. This function should:

a. Ask the user to input the module name.
b. Ask the user how many assessment components the module has.
c. For each component:
    - Ask for the component name (e.g., "Coursework 1", "Final Exam")
    - Ask for the component's weight as a percentage (ensure all weights sum to 100%)
d. Store this information in an appropriate data structure.
e. Return the module configuration.

Here's an example of how the program flow might look:

```
Welcome to the Student Grading System First, let's set up the module configuration.

Enter module name: Advanced Programming
How many assessment components does this module have? 3

Component 1 name: Midterm Exam
Component 1 weight (%): 30

Component 2 name: Final Project
Component 2 weight (%): 40

Component 3 name: Participation
Component 3 weight (%): 30

Module configuration complete.

Now, let's enter student information and grades.
Student 1:
ID: 01
Name: John Doe
Date of Birth (YYYY-MM-DD): 1995-05-15
Midterm Exam score: 75
Final Project score: 82
Participation score: 90

Results:
UID | Name     | D.o.B          | Age | Raw Score | Rounded Score | Category
-----|--------------|----------------|------ |---------------|---------------------|---------
 01 | John Doe | 1995-05-15 | 28   |82.6           | 82                  | Upper First
```

Submission Requirements: Submit your code via Codegrade as a Jupyter notebook named:

student_category.ipynb

The program file should contain full documentation explaining your implementation (comment your code). Your file should have your name, ID number and date of last update.

🔎 **Academic Integrity & Plagiarism**

All submissions will be processed through a code plagiarism tool. If signs of misconduct are found, all students involved will be contacted to discuss further steps. Please see here for information on academic integrity at the university https://portal.roehampton.ac.uk/information/Pages/Academic-Integrity.aspx.

Our guiding principle is that academic integrity and honesty are fundamental to the academic work you produce at the University of Roehampton. You are expected to complete coursework which is your own and which is referenced appropriately. The university has in place measures to detect academic dishonesty in all its forms. If you are found to be cheating or attempting to gain an unfair advantage over other students in any way, this is considered academic misconduct and you will be penalised accordingly. Please don't do it.

🏁 **Final Tips for Success**

- ✓ **Start early** – don't leave everything to the last minute.
- ✓ **Break down the problem** into smaller parts.
- ✓ **Test your code frequently** – fix errors early.
- ✓ **Use office hours** or discussion forums for help.
- ✓ **Read the coursework instructions carefully.**

📢 **Need Help?**

- 💬 **Ask your tutor or post questions on Moodle.**
- 📚 **Refer to online Python resources and documentation.**
- 👨‍💻 **Use debugging tools and test cases to refine your code.**

🔹 **Good luck! We look forward to seeing your amazing programs!** 🎯 💻 🚀

Marking Criteria

Grades will be based on number of components completed. You will be assessed on

- **Solution:** the completeness of the code, documentation, and reflection to meet the specification given.
- **Design:** ability to decompose a problem into coherent and reusable parts.
- **Correctness:** ability to create solution that reliably produces correct answers or appropriate results.
- **Logic:** ability to use correct program structures appropriate to the problem domain.
- **Clarity:** ability to format and document code for human consumption (PEP-8) and reflection.
- **Robustness:** ability of the solution to handle unexpected input and error conditions correctly as evidenced via testing.

🎓 **The following rubric will be used to assess your work:**

Basic Requirements

In order to get full marks for this rubric category your code must:

- Be encapsulated into functions including `*main()*`, `calculate_overall_score*()*`, and `determine_category*()*`. (8)
- Prompt users for an ID, name, D.o.B, and four student components, in that order. (8)
- Accept data for exactly 3 students before stopping or if "end" was entered instead of an ID. (8)

- Calculate the student' overall score. (2)
- Calculate the student' category. (2)
- Calculate the students' ages. (8)
- Display a table of all the data passed to the program using `tabulate` (8)
- Save this table to `students.txt` (8)

Intermediate requirements

In order to get full marks for the rubric category your code must:

- Implement robust input validation for a student's ID, D.o.B., and overall score (6)
- Display use of appropriate data structures for storing student information (5)
- Display use of appropriate code structures for implementing the functionality (5)

Distinction Requirements:

Distinction work in the range of 72-78 should demonstrate:

- Exemplary attainment of learning outcomes
- A high level of insight and critical evaluation of the material
- A comprehensive and up-to-date account of relevant theoretical and empirical material
- A thorough understanding and integration of material supporting a cogent argument
- Excellent writing of a high academic standard

Distinction work in the range of 82-100 should demonstrate:

- Attainment beyond the intended learning outcomes
- An outstanding level of originality and creativity, providing a significant new perspective on the question or topic
- A clear, elegant and well supported argument, based on the integration and sophisticated critical evaluation of a substantial body of knowledge
- Suitability for publication in high quality journal

Assessment Success Guidance

To **achieve high marks**, you should:

✓ Submit a **fully functional** program that meets all requirements.

✓ Write **clear, efficient, and well-structured** code.

✓ Follow **best practices** in programming (e.g., meaningful variable names, comments, modular design).

✓ Provide a **logical explanation** for your program's design, testing, and debugging approach.

Please read the Grading Rubric and bring any questions you may have to your tutor.

Assessment Guidance Support and Formative Feedback

Feedback will be given automatically through Codegrade via Moodle.

Contact for Queries/ who you can contact for further information or queries

For further information about the assessment, book a time with your tutor.

Use of Artificial Intelligence (AI)

Please read and apply:

https://roehamptonprod.sharepoint.com/sites/portal/information/LTEU/Documents/4a%20AI%20principles%20and%20guidance%20-for%20Senate.docx

https://roehamptonprod.sharepoint.com/sites/portal/information/LTEU/Documents/4b%20Student%20Guidelines%20on%20the%20use%20of%20AI_.docx

The assessment is designed so that the use of AI during the assessment is irrelevant or impossible. You are welcome to use AI to support learning and preparation for this assessment. Please make sure you read and understand the assessment guidelines and ask your Module Leader if you have any questions.
**You can find the student guidance on the use of AI in the Library and Nest*

Mitigating circumstances/late penalties

Sometimes circumstances outside of your control may affect your studies and might prevent you from submitting work on time or attending an exam.

The University offers the ability for students to request additional time to complete an assessment or to defer an examination to a later date. If you are finding yourself in such a situation, please speak to your Academic Guidance Tutor, the Roehampton Student Union (RSU) or someone in the **Wellbeing team** first, who can support you. Further details can be found on the mitigating circumstances portal.

If you do not apply for or are not approved for Mitigating Circumstances, late penalties will apply. If work is submitted up to 14 days late, the mark will be capped at 40%/50% (delete as appropriate); if it is over 14 days late, it will not be marked.

Resubmissions and Reassessment

If you are required to resubmit this assessment or take part in reassessment, you will be notified via Moodle and your student email. Please ensure you check both regularly. Any reassessment tasks will follow the same learning outcomes and criteria.

Submission Checklist

Before you submit, ask yourself:

Have I fully answered the assessment brief?

Have I met the word count and formatting requirements?

Is my referencing complete and accurate?

Have I declared any AI use honestly?

Have I proofread my work?

Am I submitting through the correct platform before the deadline?